

DBMS LAB – TASK 3

Title

Querying with Built-in SQL Functions using the Taxation Database

Objective

The objective of this laboratory experiment is to help students understand and apply **SQL Built-in Functions** for retrieving, manipulating, formatting, and converting data stored in a relational database.

By the end of this laboratory experiment, students should be able to:

- Use various **String Functions** to manipulate textual data.
 - Perform calculations using **Numeric Functions**.
 - Retrieve and manipulate dates using **Date Functions**.
 - Convert one data type into another using **Conversion Functions**.
 - Apply SQL Built-in Functions to solve practical problems using the **Taxation Database** created in Week-1.
-

Learning Outcomes

After completing this experiment, students will be able to:

- Retrieve meaningful information using SQL Built-in Functions.
- Format and manipulate citizen, application and certificate data.

- Perform numeric calculations on application fees and processing days.
 - Work efficiently with date values stored in the database.
 - Convert data between different SQL data types whenever required.
-

Pre-Requisites (Micro Topics)

Students should revise the following concepts before attending the lab.

Module 1 – Built-in String Functions

- UPPER()
 - LOWER()
 - LENGTH()
 - CONCAT()
 - SUBSTRING()
 - LEFT()
 - RIGHT()
 - REPLACE()
 - TRIM()
 - INSTR() / LOCATE()
-

Module 2 – Built-in Numeric Functions

- ROUND()
- CEIL()
- FLOOR()
- ABS()
- MOD()
- POWER()
- SQRT()

- RAND()
-

Module 3 – Date Functions

- CURDATE()
 - CURRENT_DATE()
 - NOW()
 - YEAR()
 - MONTH()
 - DAY()
 - DATE_ADD()
 - DATE_SUB()
 - DATEDIFF()
-

Module 4 – Conversion Functions

- CAST()
 - CONVERT()
-

Syntax

```
SELECT function_name(column_name)
```

```
FROM table_name;
```

For functions that require multiple parameters:

```
SELECT function_name(parameter1, parameter2, ...)
```

```
FROM table_name;
```

For conversion functions:

```
SELECT CAST(column_name AS datatype)
```

```
FROM table_name;
```

```
SELECT CONVERT(column_name, datatype)
```

```
FROM table_name;
```

Note: Replace `function_name`, `column_name`, `table_name`, `parameter1`, `parameter2`, and `datatype` according to the requirements of each task.

Software Required (ignore if already installed)

- MySQL Server 8.x
 - MySQL Workbench
 - Git
 - GitHub Account
 - Week-1 Gram Panchayat Database
-

Database to be Used

Continue using the database created in Week-1.

taxation_db

Tables already available

- Taxpayer

- Income_Category
- Financial_Year
- Income_Record

No new tables need to be created.

If required, add more rows or additional sample data to perform the following tasks.

Laboratory Activities

Part A – Verify Week-1 Database

Before starting the experiment,

Open MySQL Workbench.

Select the database.

```
USE taxation_db;
```

Display all tables.

```
SHOW TABLES;
```

Verify that all four tables contain the inserted records.

Part B – Built-in String Functions

Level 1 (Understanding)

1. Display all taxpayer names in uppercase.
2. Display all occupations in lowercase.
3. Find the length of each taxpayer's name.
4. Display the first four characters of every PAN number (or Taxpayer ID if PAN is unavailable).
5. Concatenate the taxpayer name with occupation.

Example Output

Ravi Kumar - Software Engineer

Anitha Rao - Teacher

Level 2 (Application)

6. Replace the word **Income** with **Inc.** in income category names.

Example

Business Inc.

Salary Inc.

Rental Inc.

7. Remove leading or trailing spaces from taxpayer names.
 8. Display only the first name of every taxpayer.
-

Level 3 (Advanced)

9. Generate a display string in the following format.

Taxpayer : Ravi Kumar

Occupation : Software Engineer

10.Display all taxpayers whose PAN number begins with **AP**.

Part C – Built-in Numeric Functions

Level 1

- 1.Round every annual income.
 - 2.Display the absolute value of annual_income - 500000
 - 3.Display the square of annual income.
-

Level 2

- 4.Find the remainder when annual income is divided by **1000**.
 - 5.Round annual income to two decimal places.
 - 6.Display the ceiling and floor values of annual income.
-

Level 3

- 7.Generate a random integer between **1 and 100**.
- 8.Display the square root of annual income.

9. Calculate $\text{annual_income} \times 1.10$ for every taxpayer to estimate income after a 10% increment.

Part D – Date Functions

Level 1

1. Display today's date.
 2. Display current date and time.
 3. Display only the year from every financial year start date.
 4. Display only the month from every financial year start date.
 5. Display only the day of the month.
-

Level 2

6. Display the financial year end date by adding one year to the start date.
 7. Display the financial year start date after adding **30 days**.
 8. Display the financial year start date after subtracting **7 days**.
-

Level 3

9. Calculate the number of days between today's date and the financial year start date.
10. Display financial years belonging to the current year.

Part E – Conversion Functions

Level 1

- 1.Convert annual income into INTEGER.
 - 2.Convert taxpayer ID into CHAR.
-

Level 2

- 3.Convert the financial year start date into DATETIME.
 - 4.Convert annual income into DECIMAL.
-

Level 3

- 5.Display annual income as character strings.
 - 6.Convert numeric values before performing arithmetic operations to calculate tax.
-

Time-wise Laboratory Plan

Time	Activity
0–30 min	Verify Week-1 database and revise SQL Built-in Functions

30–60 min	Practice String Functions
60–90 min	Practice Numeric Functions
90–120 min	Practice Date Functions
120–150 min	Practice Conversion Functions and Challenge Exercise
150–180 min	GitHub Submission, Faculty Verification and Discussion

GitHub Submission

Students should create the following folder structure.

```
StudentID-repo
├── Week-3-Lab
│   ├── week3_functions.sql
│   ├── screenshots.pdf(ignore)
│   └── report.pdf
```

Submission Requirements

Each student must submit:

1. SQL File

week3_functions.sql

containing all SQL queries.
